
```

% solution to assignment 4_1(i)
function ass4_1a
% numerical solution using runge_kutta
f=@cstr;
t=[0:0.1:3];
c0=0;
c_rk=runge_kutta(f,t,c0);
% analytical solution using dsolve
syms V F Q k c t;
c=dsolve('Dc = (F-Q*c-k*V*c^2)/V','c(0) = 0','t');
V=10; F=200; Q=1; k=0.1;
c=subs(c);
t=[0:0.1:3];
c_an=subs(c);
plot(t,c_an,'-bo',t,c_rk,'-rs');
legend('analytical','numerical');
xlabel('time (min)');
ylabel('concentration (g/m^3)');
return

```

```

function [y] = runge_kutta(f,x,y0)
n=length(x); m=length(y0);
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    dx=x(i+1)-x(i);
    k1=feval(f,x(i),y(:,i));
    k2=feval(f,x(i)+dx/2,y(:,i)+k1*dx/2);
    k3=feval(f,x(i)+dx/2,y(:,i)+k2*dx/2);
    k4=feval(f,x(i)+dx,y(:,i)+k3*dx);
    y(:,i+1)=y(:,i)+dx*(k1+2*k2+2*k3+k4)/6;
end
return

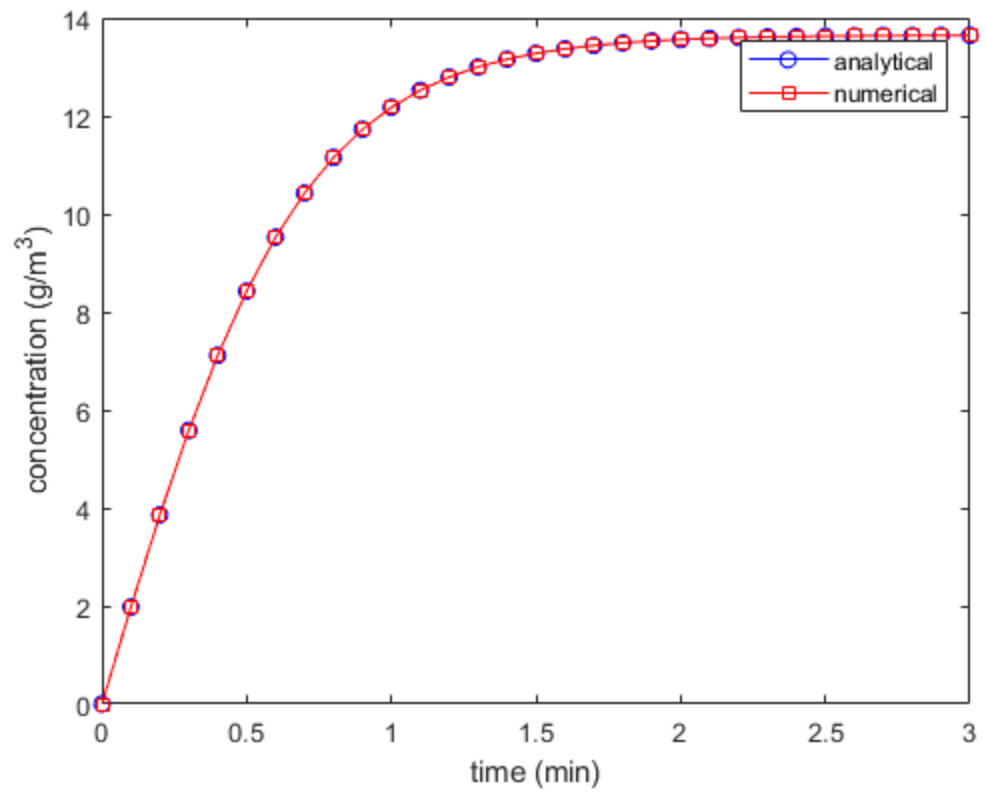
```

```

function [dcdt] = cstr(t,c)
V=10; F=200; Q=1; k=0.1;
dcdt(1)=(F-Q*c-k*V*c^2)/V;
return

```

Warning: Support of character vectors and strings will be removed in a future release. Use sym objects to define differential equations instead.



Published with MATLAB® R2020a

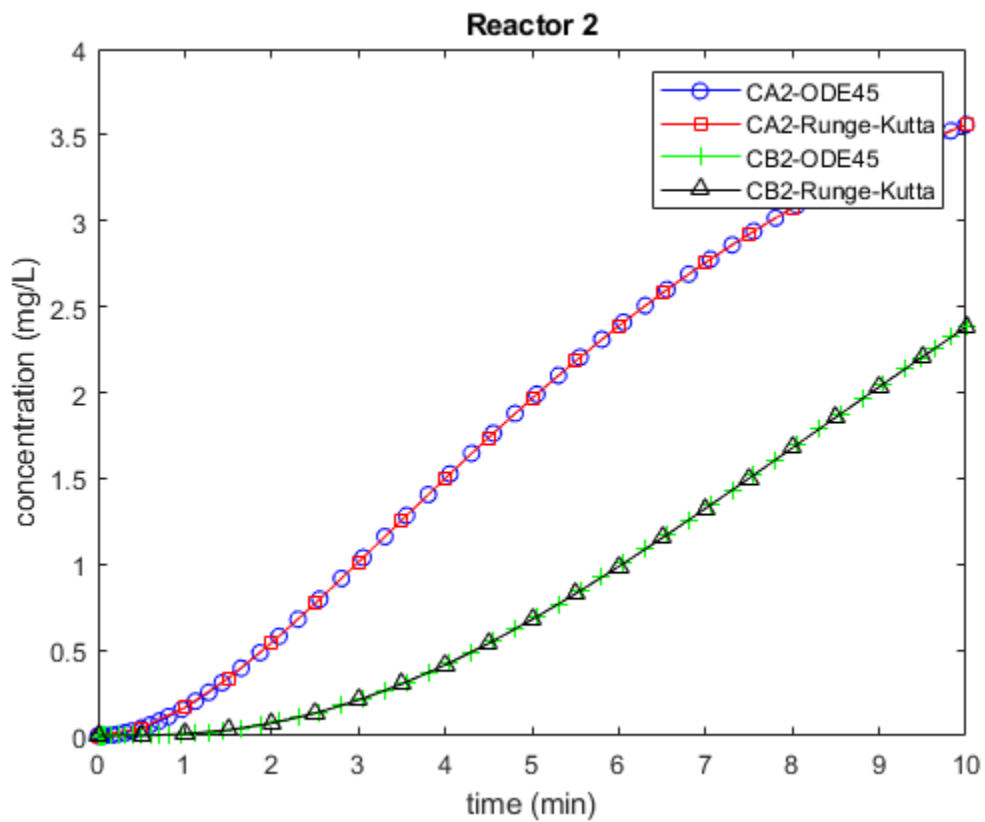
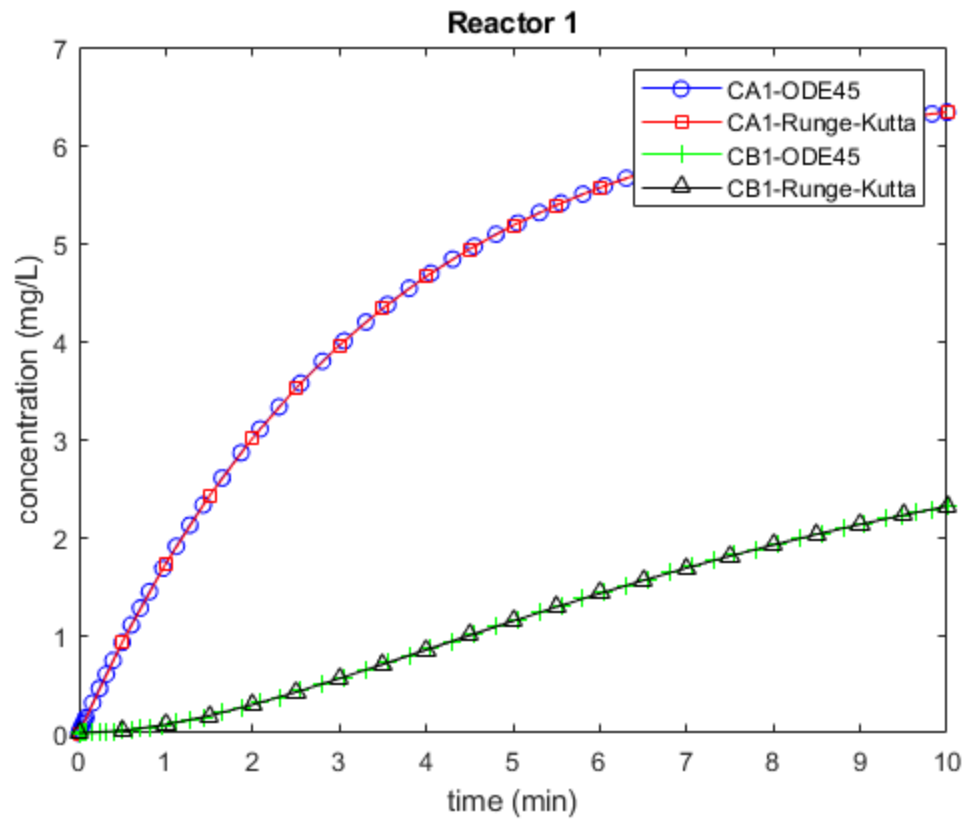
```

% solution to problem 4_1b
function ass4_1b
% numerical solution using our own runge_kutta
f=@series_reactor;
t=0:0.5:10;
c0=[0,0,0,0];
c_rk=runge_kutta(f,t,c0);
% numerical solution using MATLAB runge_kutta
[T,c_ode45]=ode45(f,[0,10],c0);
figure(1);
plot(T,c_ode45(:,1),'-bo',t,c_rk(1,:),'-rs',T,c_ode45(:,2),'-g
+',t,c_rk(2,:),'-k^');
legend('CA1-ODE45','CA1-Runge-Kutta','CB1-ODE45','CB1-Runge-Kutta');
title('Reactor 1');
xlabel('time (min)');
ylabel('concentration (mg/L)');
figure(2);
plot(T,c_ode45(:,3),'-bo',t,c_rk(3,:),'-rs',T,c_ode45(:,4),'-g
+',t,c_rk(4,:),'-k^');
legend('CA2-ODE45','CA2-Runge-Kutta','CB2-ODE45','CB2-Runge-Kutta');
title('Reactor 2');
xlabel('time (min)');
ylabel('concentration (mg/L)');
return

function [y] = runge_kutta(f,x,y0)
n=length(x); m=length(y0);
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    dx=x(i+1)-x(i);
    k1=feval(f,x(i),y(:,i));
    k2=feval(f,x(i)+dx/2,y(:,i)+k1*dx/2);
    k3=feval(f,x(i)+dx/2,y(:,i)+k2*dx/2);
    k4=feval(f,x(i)+dx,y(:,i)+k3*dx);
    y(:,i+1)=y(:,i)+dx*(k1+2*k2+2*k3+k4)/6;
end
return

% cstr function
function [dcdt] = series_reactor(t,c)
CA0=10; k=0.1; tau=5;
CA1=c(1); CB1=c(2); CA2=c(3); CB2=c(4);
dcdt=zeros(4,1); % MATLAB's ode45 requirement!
tau_inv=1/tau;
dcdt(1)= tau_inv*(CA0-CA1)-k*CA1;
dcdt(2)= -tau_inv*CB1+k*CA1;
dcdt(3)= tau_inv*(CA1-CA2)-k*CA2;
dcdt(4)= tau_inv*(CB1-CB2)+k*CA2;
return

```

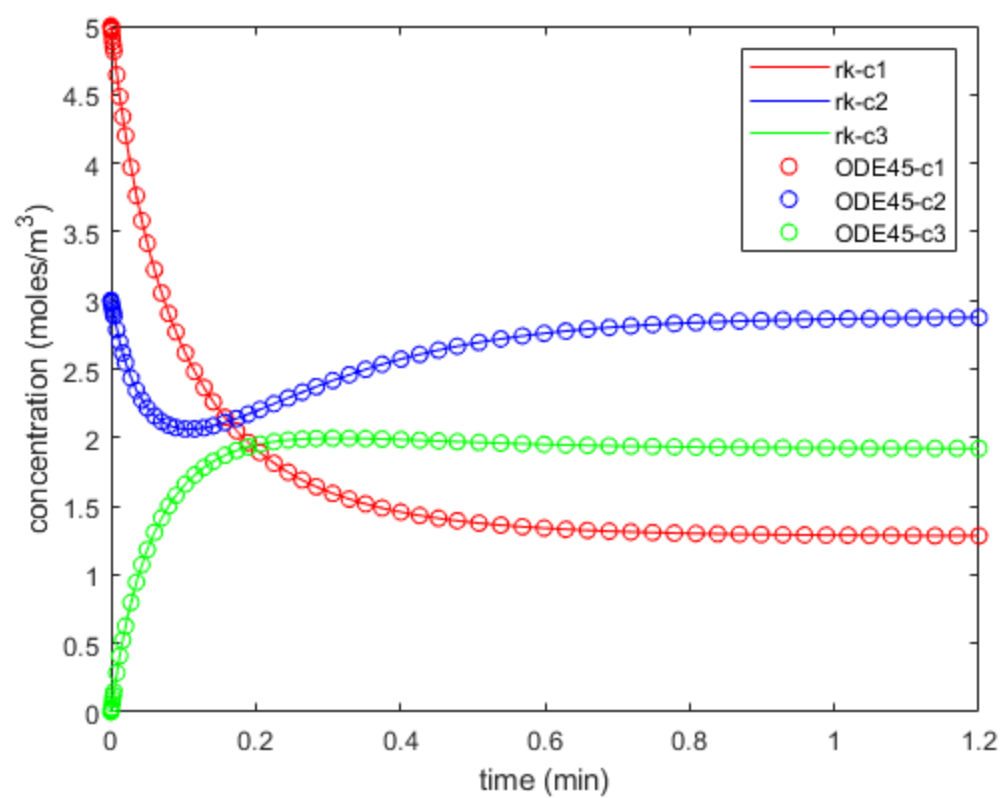


Published with MATLAB® R2020a

```
% cstr script
% cstr problem
function ass4_2
c0 = [5 3 0]'; % c0 needs to be column vector in ode_rk4
t = linspace(0,1.2,101);
f=@cstr;
[c] = runge_kutta(f,t,c0);
% solution using ode45
[T,C]=ode45(f,[0,1.2],c0);
% plot results
plot(t,c(1,:), 'r', t,c(2,:), 'b', t,c(3,:), 'g', T,C(:,1), 'ro', T,C(:,2), 'bo', T,C(:,3), 'r');
xlim([0 1.2]);
legend('rk-c1', 'rk-c2', 'rk-c3', 'ODE45-c1', 'ODE45-c2', 'ODE45-c3');
xlabel('time (min)');
ylabel('concentration (moles/m^3)');
hold off;
return

% cstr function
function [dcdt] = cstr(t,c)
c01=3.2; c02=4.8;
q=10;V=2; alpha1=2.6; alpha2=2.6;
alpha3=2.6;
b=q/V;
dcdt=zeros(3,1); % MATLAB's ode45 requirement!
dcdt(1)= b*(c01-c(1))-alpha1*c(1)*c(2);
dcdt(2)= b*(c02-c(2))-alpha2*c(1)*c(2);
dcdt(3)= -b*c(3)+alpha3*c(1)*c(2);
return

function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = feval(df,xi,yi);
    k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);
    k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
    k4 = feval(df,xi+h,yi+k3*h);
    y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;
end
return
```



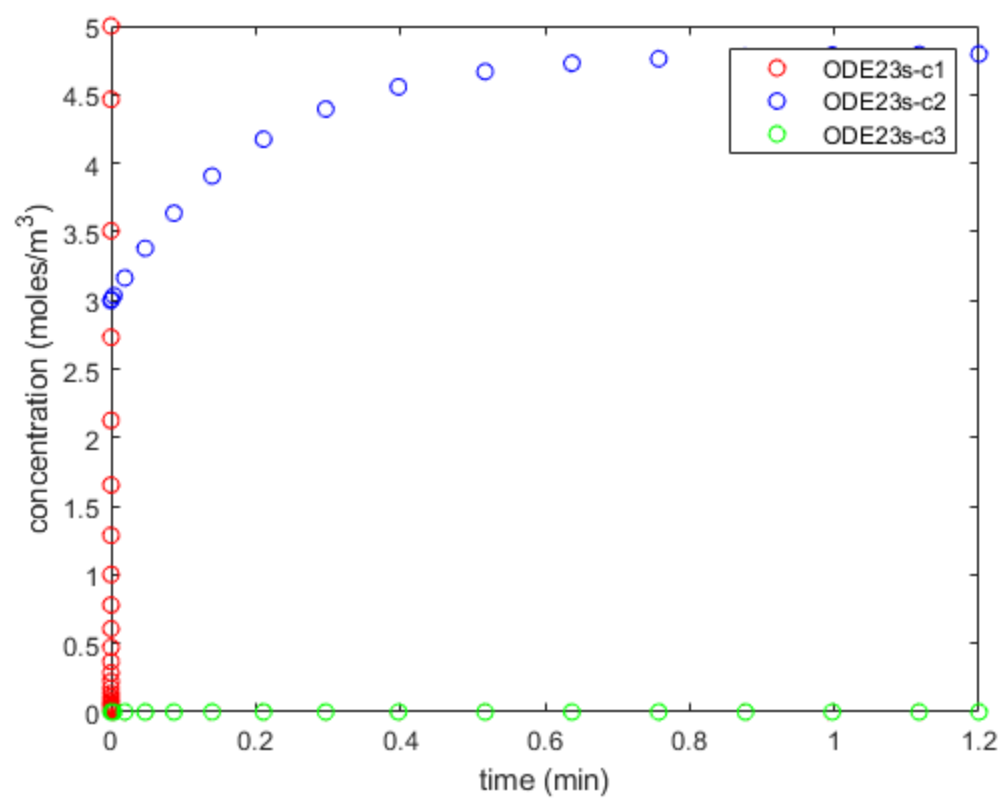
Published with MATLAB® R2020a

```
% cstr script
% cstr problem
function ass4_2b
tic;
c0 = [5 3 0]'; % c0 needs to be column vector in ode_rk4
% t = linspace(0,1.2,101);
% [c] = runge_kutta('cstr',t,c0);
% solution using ode45
[T,C]=ode23s(@cstr,[0,1.2],c0);
% plot results
plot(T,C(:,1),'ro',T,C(:,2),'bo',T,C(:,3),'go');
xlim([0 1.2]);
legend('ODE23s-c1','ODE23s-c2','ODE23s-c3');
xlabel('time (min)');
ylabel('concentration (moles/m^3)');
hold off;
toc;
return

% cstr function
function [dcdt] = cstr(t,c)
c01=3.2; c02=4.8;
q=10;V=2;
alpha1=100000; alpha2=1; alpha3=10;
b=q/V;
dcdt=zeros(3,1); % MATLAB's ode45 requirement!
dcdt(1)= b*(c01-c(1))-alpha1*c(1)*c(2);
dcdt(2)= b*(c02-c(2))-alpha2*c(1)*c(2);
dcdt(3)= -b*c(3)+alpha3*c(1)*c(2);
return

function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = feval(df,xi,yi);
    k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);
    k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
    k4 = feval(df,xi+h,yi+k3*h);
    y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;
end
return
```

Elapsed time is 0.089400 seconds.



Published with MATLAB® R2020a

```

% solution to problem 4_3
function ass4_3
f=@batch_reactor;
t=0:0.1:10;
y0=[1,15];
% numerical solution using base runge_kutta
y_rk=runge_kutta(f,t,y0);
% numerical solution using fully implicit
y_fi=fully_implicit(f,t,y0);
% numerical solution using MATLAB runge_kutta
[T,y_ode45]=ode45(f,[0,10],y0);
% plot results
figure(1);
plot(T,y_ode45(:,1),'-b',t,y_rk(1,:), 'rs',t,y_fi(1,:), 'go');
legend('ODE45', 'Runge-Kutta', 'Fully Implicit');
xlabel('time (t)');
ylabel('concentraion (mg/L)');
figure(2);
plot(T,y_ode45(:,2),'-b',t,y_rk(2,:), 'rs', t, y_fi(2,:), 'go');
legend('ODE45', 'Runge-Kutta', 'Fully Implcit');
xlabel('time(t)');
ylabel('temperature (^oC)');
return

function [y] = runge_kutta(f,x,y0)
n=length(x); m=length(y0);
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    dx=x(i+1)-x(i);
    k1=feval(f,x(i),y(:,i));
    k2=feval(f,x(i)+dx/2,y(:,i)+k1*dx/2);
    k3=feval(f,x(i)+dx/2,y(:,i)+k2*dx/2);
    k4=feval(f,x(i)+dx,y(:,i)+k3*dx);
    y(:,i+1)=y(:,i)+dx*(k1+2*k2+2*k3+k4)/6;
end
return

% cstr function
function [dydt] = batch_reactor(t,y)
dydt=zeros(2,1); % MATLAB's ode45 requirement!
C=y(1); T=y(2);
dydt(1)= -exp(-10/(T+273))*C;
dydt(2)= 1000*exp(-10/(T+273))*C-10*(T-20);
return

function [y] = fully_implicit(f,t,y0)
n = length(t);
m=length(y0);
y=zeros(m,n);
y(:,1)=y0;
I=eye(m);

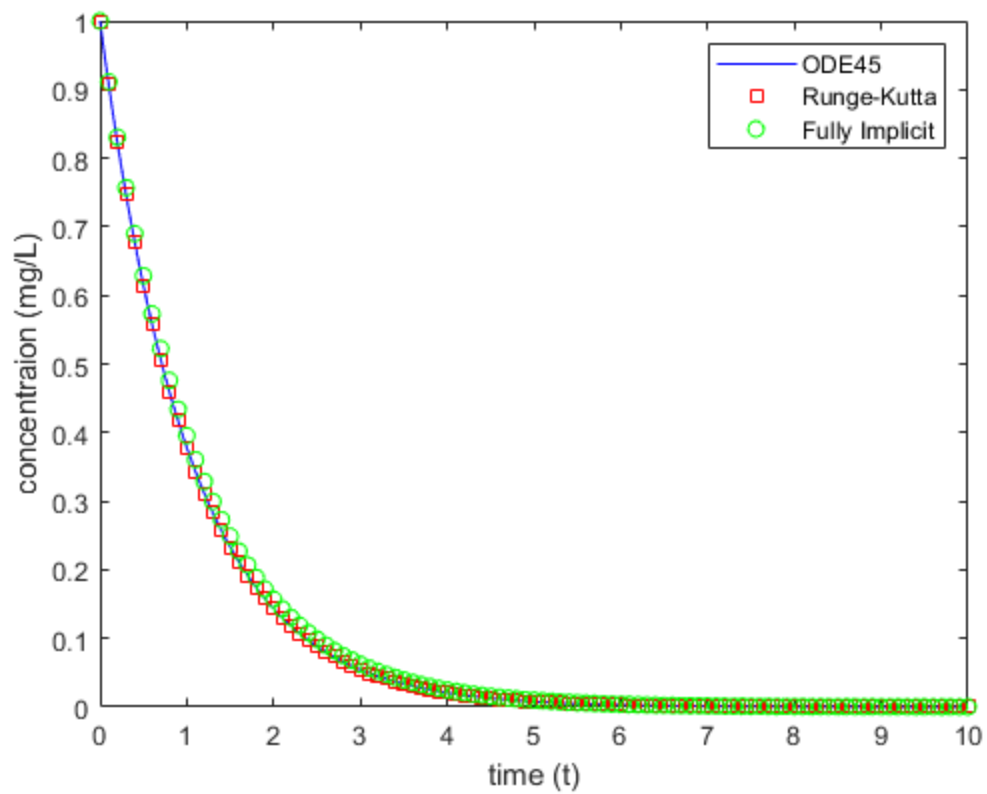
```

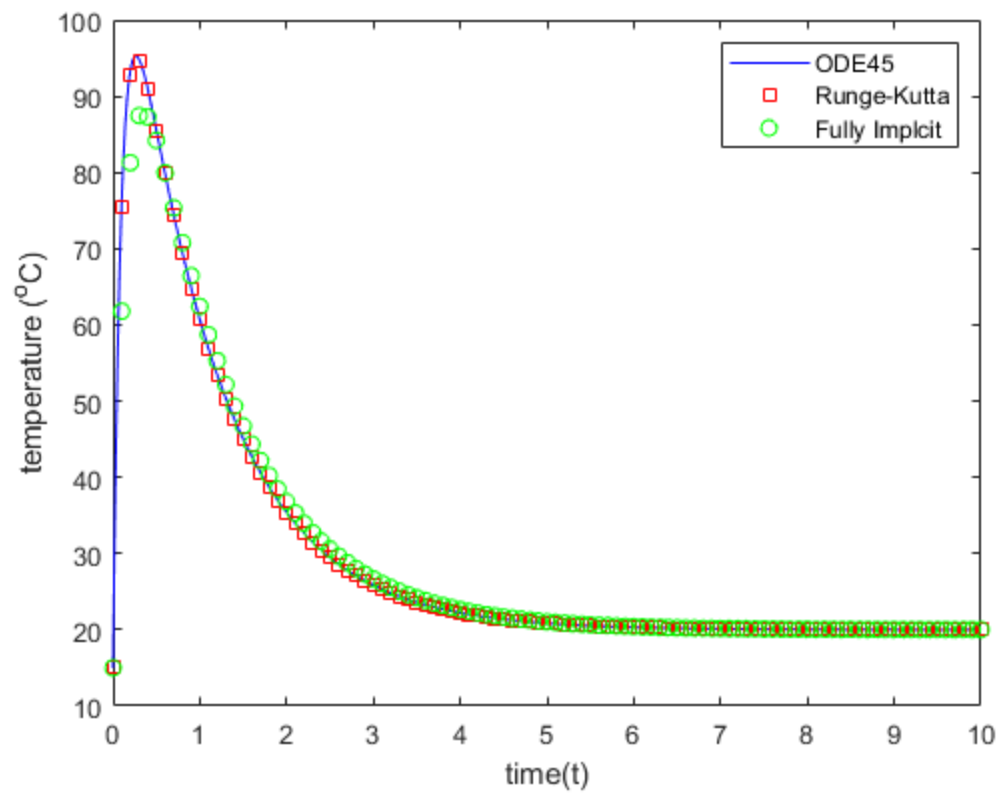
```

for i=1:n-1
    h = t(i+1)-t(i);
    J = feval('my_jacobian',f,t(i+1),y(:,i));
    dy = h*inv(I-h*J)*feval(f,t(i+1),y(:,i));
    y(:,i+1) = y(:,i)+dy;
end
return

function [J] = my_jacobian(f,t,y)
m=length(y);
h=1e-3;
for i = 1:m
    y1 = y;
    y2 = y;
    y1(i) = y(i)+h;
    y2(i) = y(i)-h;
    J(:,i) = (feval(f,t,y1)-feval(f,t,y2))/(2*h);
end
return

```



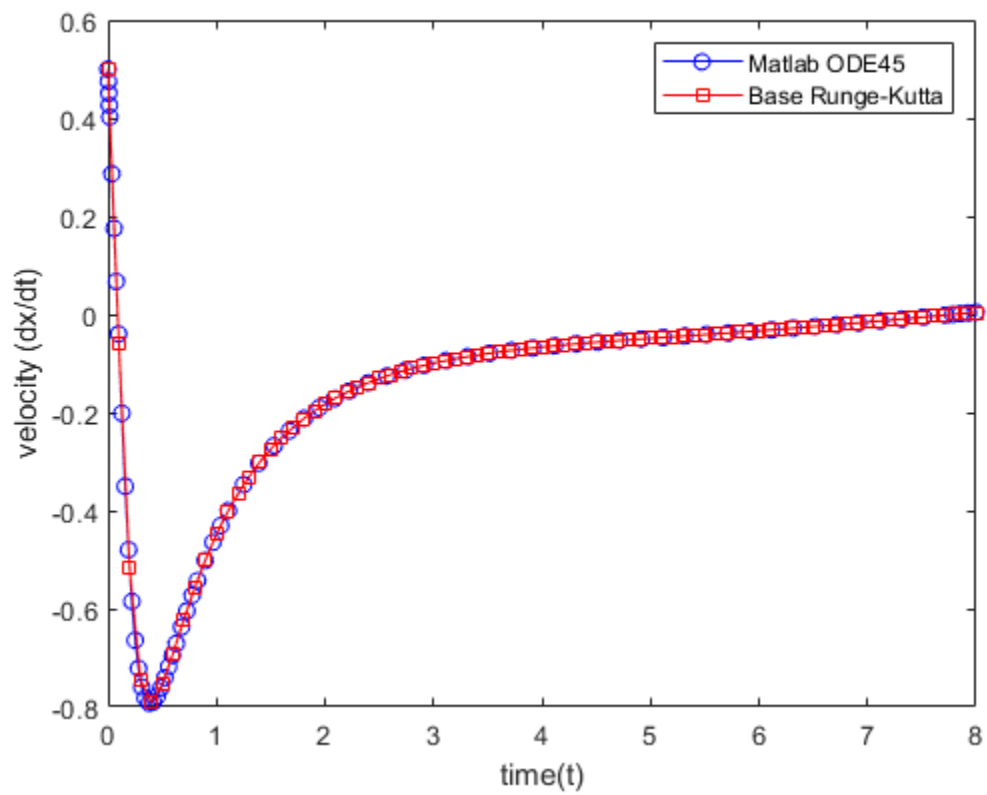
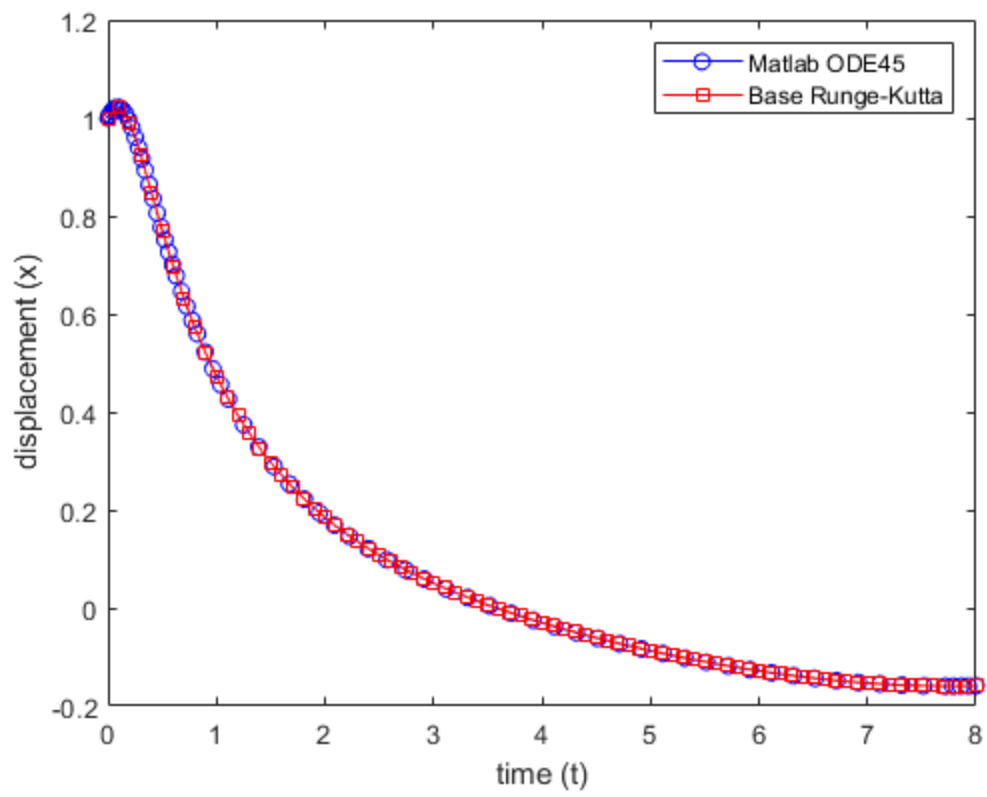


Published with MATLAB® R2020a

```
% solution to problem 4_4
function ass4_4
% numerical solution using our own runge_kutta
f=@spring;
t=[0:0.1:8];
y0=[1,0.5];
y_rk=runge_kutta(f,t,y0);
% numerical solution using MATLAB runge_kutta
[T,y_ode45]=ode45(f,[0,8],y0);
% plot results
figure(1);
plot(T,y_ode45(:,1),'-bo',t,y_rk(1,:),'-rs');
legend('Matlab ODE45','Base Runge-Kutta');
xlabel('time (t)');
ylabel('displacement (x)');
figure(2);
plot(T,y_ode45(:,2),'-bo',t,y_rk(2,:),'-rs');
legend('Matlab ODE45','Base Runge-Kutta');
xlabel('time(t)');
ylabel('velocity (dx/dt)');
return

function [y] = runge_kutta(f,x,y0)
n=length(x); m=length(y0);
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    dx=x(i+1)-x(i);
    k1=feval(f,x(i),y(:,i));
    k2=feval(f,x(i)+dx/2,y(:,i)+k1*dx/2);
    k3=feval(f,x(i)+dx/2,y(:,i)+k2*dx/2);
    k4=feval(f,x(i)+dx,y(:,i)+k3*dx);
    y(:,i+1)=y(:,i)+dx*(k1+2*k2+2*k3+k4)/6;
end
return

% cstr function
function [dydt] = spring(t,y)
m=1; a=5; b=5;
dydt=zeros(2,1); % MATLAB's ode45 requirement!
dydt(1)= y(2);
dydt(2)= -(a*y(2)^2*sign(y(2))+b*y(1)^3)/m;
return
```

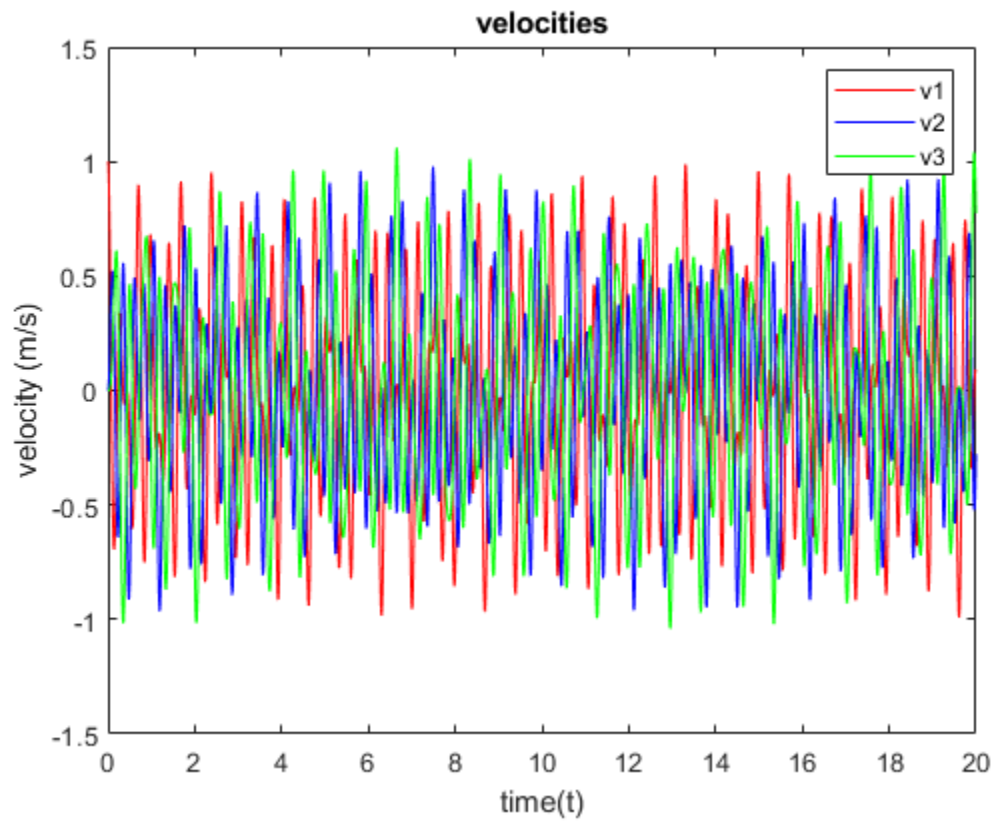
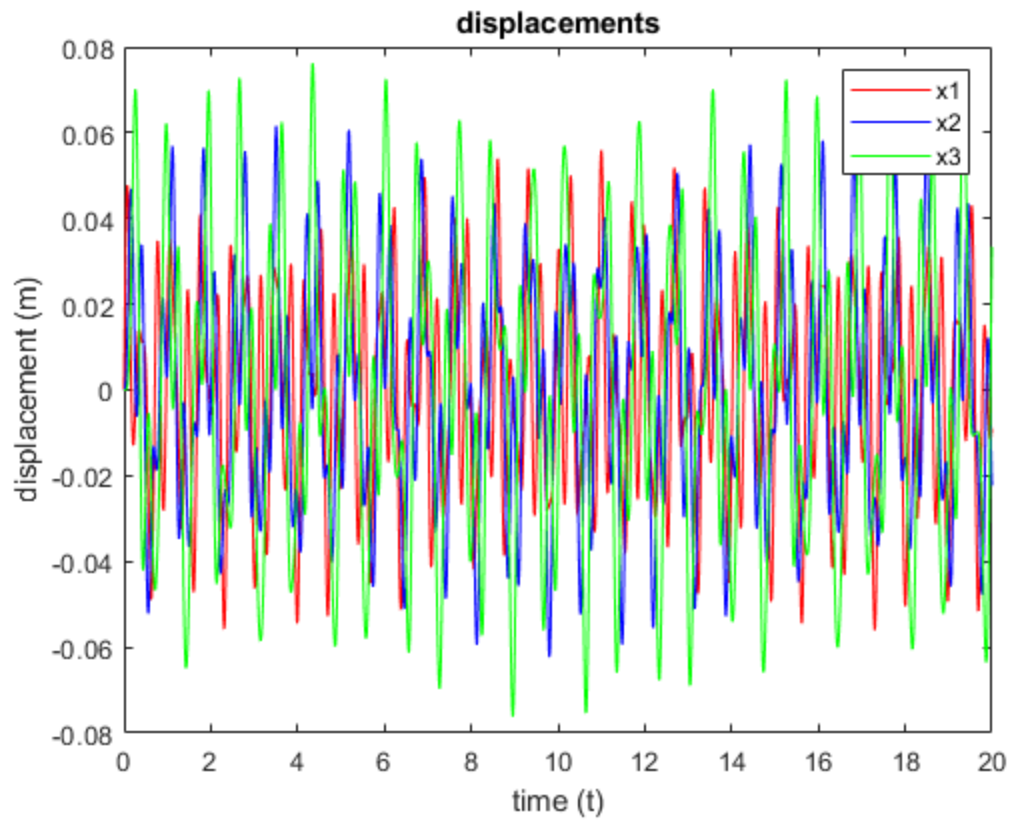


Published with MATLAB® R2020a

```
% solution to problem 4_5
function ass4_5
% numerical solution using runge_kutta
f=@mass_spring;
t=0:0.01:20;
y0=[0 1 0 0 0 0];
y=runge_kutta(f,t,y0);
% plot results
figure(1);
plot(t,y(1,:), '-r', t,y(3,:), '-b', t,y(5,:), '-g');
legend('x1', 'x2', 'x3');
xlabel('time (t)');
ylabel('displacement (m)');
title('displacements');
figure(2);
plot(t,y(2,:), '-r', t,y(4,:), '-b', t,y(6,:), '-g');
legend('v1', 'v2', 'v3');
xlabel('time(t)');
ylabel('velocity (m/s)');
title('velocities');
return

function [y] = runge_kutta(f,x,y0)
n=length(x); m=length(y0);
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    dx=x(i+1)-x(i);
    k1=feval(f,x(i),y(:,i));
    k2=feval(f,x(i)+dx/2,y(:,i)+k1*dx/2);
    k3=feval(f,x(i)+dx/2,y(:,i)+k2*dx/2);
    k4=feval(f,x(i)+dx,y(:,i)+k3*dx);
    y(:,i+1)=y(:,i)+dx*(k1+2*k2+2*k3+k4)/6;
end
return

% spring mass function
function [dydt] = mass_spring(t,y)
dydt=zeros(6,1); % MATLAB's ode45 requirement!
k1=3000*1e3; k2=2400*1e3; k3=1800*1e3;
m1=12000; m2=10000; m3=8000;
x1=y(1); x2=y(3); x3=y(5);
dydt(1)= y(2);
dydt(2)= -k1*x1/m1+k2*(x2-x1)/m1;
dydt(3)= y(4);
dydt(4)= k2*(x1-x2)/m2+k3*(x3-x2)/m2;
dydt(5) = y(6);
dydt(6) = k3*(x2-x3)/m3;
return
```



Published with MATLAB® R2020a

```

% assignment 4 problem 6
function ass4_6
% cantilever beam problem
y0 = [0 0]';
x=0:0.01:2;
n=length(x);
[y] = runge_kutta('cantilever',x,y0);
fprintf('Base Runge_Kutta function: Deflection at L = 2 m is %g cm\n', -y(1,n)*100);
% solution using ode45
[X,Y]=ode45(@cantilever,[0 2],y0);
plot(x,y(1,:), 'r', x,y(2,:), 'b', X,Y(:,1), '-ro', X,Y(:,2), '-bo');
xlim([0 2]);
legend('numerical-y1', 'numerical-y2', 'matlab-y1', 'matlab-y2');
xlabel('x (m)');
ylabel('deflection (m)');
n = length(Y(:,1));
fprintf('Matlab ODE45 function: Deflection at L = 2 m is %g cm\n', -Y(n,1)*100);
hold off;
return

```

```

% cantilever beam function
function [dydx] = cantilever(x,y)
g=9.8; L =2; JE = 2400; rho = 10;
dydx=zeros(2,1);
dydx(1)= y(2);
dydx(2)= -rho*g*(1+y(2)^2)^1.5*(x*(x-L/2)+L^2/2)/JE;
return

```

```

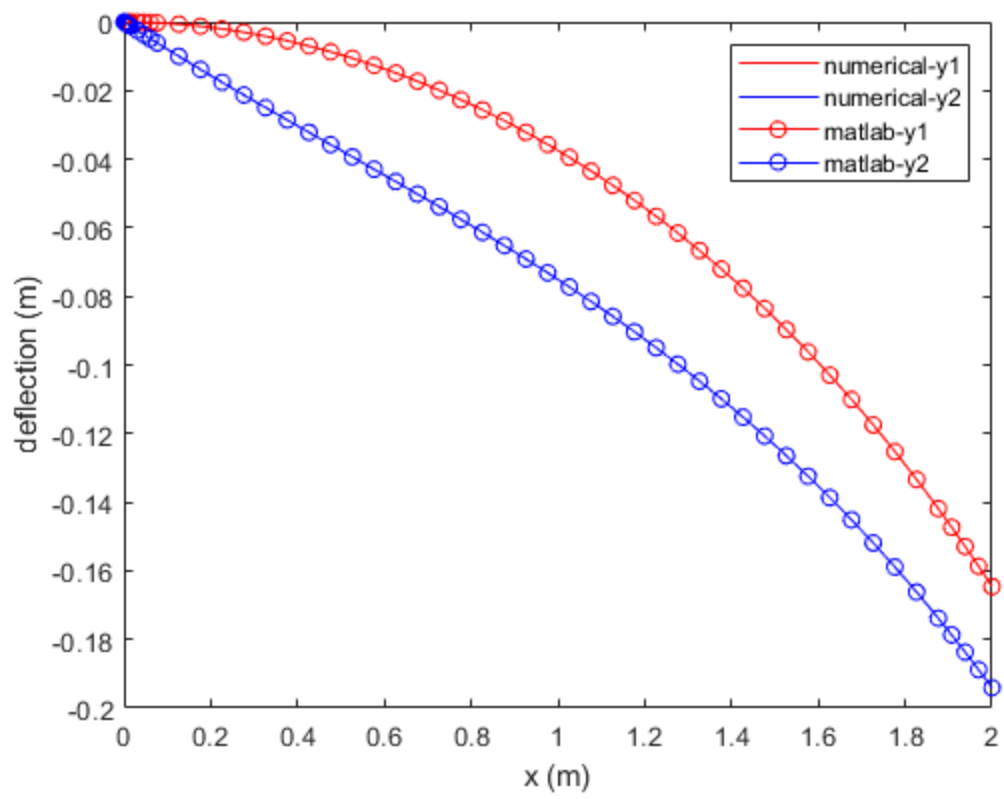
function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = feval(df,xi,yi);
    k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);
    k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
    k4 = feval(df,xi+h,yi+k3*h);
    y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;
end
return

```

```

Base Runge_Kutta function: Deflection at L = 2 m is 16.4617 cm
Matlab ODE45 function: Deflection at L = 2 m is 16.4617 cm

```



Published with MATLAB® R2020a

```
% assignment 4_7 part (i) and (ii)
function ass4_7a
% matlab bvp4c
xinit=[0:0.1:5];
yinit=[0 0];
solinit = bvpinit(xinit,yinit);
sol = bvp4c(@beam, @beam_bc, solinit);
xint=xinit;
yint = deval(sol,xint);
% shooting method
y0=0;
yL=0;
x=[0:0.1:5];
y=bvp_shooting(@beam,x,y0,yL);
% fda
yfda=bvp_fda_beam(x,y0,yL);
% analytical solution
ya = beam_analytical(x);
% plot all
plot(xint,yint(1,:), 'ro', x,y(1,:), 'bs', x,yfda, 'g+', x,ya, '-k');
legend('matlab-bvp4c', 'shooting', 'fda', 'analytical');
xlabel('length x (m)');
ylabel('deflection y (m)');
title('Simply supported beam: small deflection case');
hold off;
return

% beam bc function for bvp4c
function [res] = beam_bc(ya,yb)
res = [ya(1); yb(1)];
return

% beam function
function [dydx] = beam(x,y)
L = 5; E=20e9; w=0.05; h=0.1; q=1500;
I=w*h^3/12;
dydx=zeros(2,1);
dydx(1)= y(2);
dydx(2)= -q*(x^2-L*x)/(2*E*I);
return

function [y] = beam_analytical(x)
L = 5; q = 1500; E = 2e10; w = 0.05; h = 0.1;
I = w*h^3/12;
y = -q*(x.^4-2*L*x.^3+L^3*x)/(24*E*I);
return

function [y] = bvp_fda_beam(x,y0,yL)
% beam problem
L = 5; q = 1500; E = 2e10; w = 0.05; h = 0.1;
```

```

n=length(x);
I = w*h^3/12;
dx=x(2)-x(1);
qEI=q/(2*E*I);
b = qEI*(x'./^2-L*x')*dx^2;
d = -ones(n,1);
e = 2*ones(n,1);
f = -ones(n,1);
d(1)=0; d(n)=0;
e(1)=1; e(n)=1;
f(1)=0; f(n)=0;
b(1)=y0;
b(n)=yL;
A = my_spdiags([d e f],[-1 0 1]);
% spy(A);
y = A\b;
return

function [y] = bvp_shooting(odefun,x,yi,yf)
n=length(x);
x0=x(1);
xL=x(n);
y0=zeros(2,1);
% first shot
y0(1)=yi;
y0(2)=1; % assume
yi1=y0(2);
y=runge_kutta(odefun,x,y0);
yL1=y(1,n);
if (yL1 < yf)
    y0(2)=y0(2)+5;
else
    y0(2)=y0(2)-5;
end
yi2=y0(2);
% second shot
y=runge_kutta(odefun,x,y0);
yL2=y(1,n);
y0(2)=yi1+(yf-yL1)*(yi2-yi1)/(yL2-yL1);
y=runge_kutta(odefun,x,y0);
return

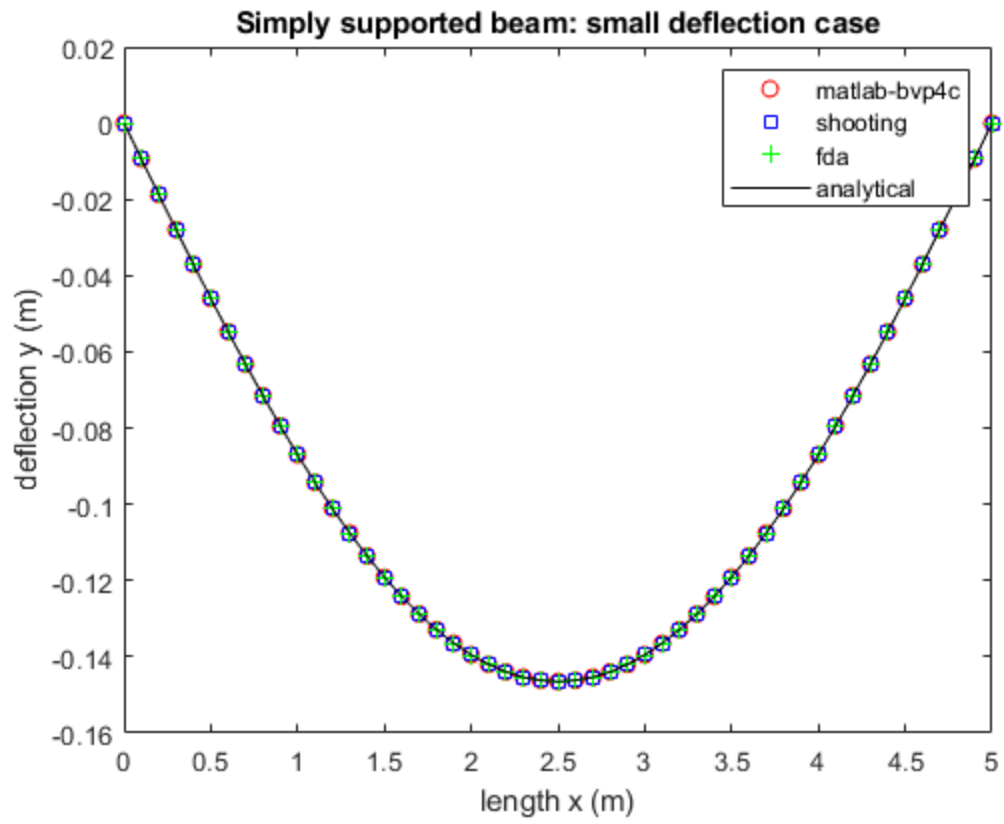
function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = feval(df,xi,yi);
    k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);

```

```

        k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
        k4 = feval(df,xi+h,yi+k3*h);
        y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;
    end
    return
% modification of spdiags to adopt linpack convention
% lower diagonals taken from bottom
% upper diagonals taken from top
% i.e., rows of diagonals directly correspond to rows of matrix
% matrix is assumed to be square
% input A = a tall nxm matrix containing the 'm' diagonals
% input id = a 1xm row vector containing the 'm' offsets for each
% diagonal
% output A = nxn sparse matrix
function [A] = my_spdiags(A,id)
[n m] = size(A);
B = zeros(n,m);
for i=1:m
    if (id(i) <= 0)
        B(1:n+id(i),i) = A(-id(i)+1:n,i);
    else
        B(1+id(i):n,i) = A(1:n-id(i),i);
    end
end
% construct matrix
A = spdiags(B,id,n,n);
return

```



Published with MATLAB® R2020a

```

% assignment 4 part (iii) and (iv)
function ass4_7b
% matlab bvp4c
x=0:0.1:5;
y0=0; yL=0;
yinit=[y0 yL];
solinit = bvpinit(x,yinit);
sol = bvp4c(@beam_n1, @beam_bc, solinit);
y_bvp4c = deval(sol,x);
% shooting method
dy0=0;
y_shoot=bvp_shooting(@beam_n1,x,y0,yL,dy0);
% fda
y_fda=beam_newton_fda(x,y0,yL);
% plot all
plot(x,y_bvp4c(1,:), '-ro', x,y_shoot(1,:), '-bs', x,y_fda, '-g^');
legend('matlab-bvp4c', 'shooting', 'fda');
xlabel('length x (m)');
ylabel('deflection y (m)');
title('Simply supported beam: large deflection case');
hold off;
return

% beam bc function for bvp4c
function [res] = beam_bc(ya,yb)
res = [ya(1); yb(1)];
return

% beam function
function [dydx] = beam_n1(x,y)
L =5; E=20e9; w=0.05; h=0.1; q=1500;
I=h*w^3/12;
dydx=zeros(2,1);
dydx(1)= y(2);
dydx(2)= -q*((1+y(2)^2)^1.5)*(x^2-L*x)/(2*E*I);
return

function [y] = bvp_shooting(odefun,x,yi,yf,dy0)
n=length(x);
y0=zeros(2,1);
y0(1)=yi;
y0(2) = dy0; % initial guess for y0(2)
y0(2)=fsolve(@bvp_shoot,dy0,optimset('fsolve'),odefun,x,yi,yf);
y=runge_kutta(odefun,x,y0);
return

function [res] = bvp_shoot(dy0,odefun,x,yi,yf)
n=length(x);

```

```

x0=x(1);
xL=x(n);
y0=zeros(2,1);
y0(1)=yi;
y0(2)=dy0;
y=runge_kutta(odefun,x,y0);
res=yf-y(1,n);
return

```

```

function [y] = runge_kutta(df,x,y0)
m = length(y0);
n = length(x);
% y is a matrix of size 'mxn'
y=zeros(m,n);
y(:,1)=y0;
for i=1:n-1
    h = x(i+1)-x(i);
    xi=x(i);
    yi=y(:,i);
    k1 = feval(df,xi,yi);
    k2 = feval(df,xi+0.5*h,yi+0.5*k1*h);
    k3 = feval(df,xi+0.5*h,yi+0.5*k2*h);
    k4 = feval(df,xi+h,yi+k3*h);
    y(:,i+1) = y(:,i) + h*(k1+2*k2+2*k3+k4)/6;
end
return

```

```

function [y] = beam_newton_fda(x,y0,yL)
maxitr=100;
m=length(x);
n=m-2;
yk=zeros(n,1);
xk=x(2:m-1);
y=zeros(n,1);
epsilon=inf;
iter=0;
f=@beam_df;
params={xk,y0,yL};
while (epsilon > 1e-3 && iter <= maxitr)
    E=feval(f,yk,params);
    J=my_jacobian(f,yk,params); % jacobian
    dy=-J\E; % correction vector
    epsilon=norm(dy,inf);
    yk=yk+dy;
    iter=iter+1;
end
y(2:m-1)=yk(1:n);
y(1)=y0;
y(m)=yL;
fprintf('number of iterations = %g\n',iter);
fprintf('residual error = %g\n',epsilon);
return

```

```

function [J] = my_jacobian(f,y,params)
m=length(y); h=1e-3; J=zeros(m);
for i = 1:m
    y1 = y;
    y2 = y;
    y1(i) = y(i)+h;
    y2(i) = y(i)-h;
    J(:,i) = (feval(f,y1,params)-feval(f,y2,params))/(2*h);
end
return

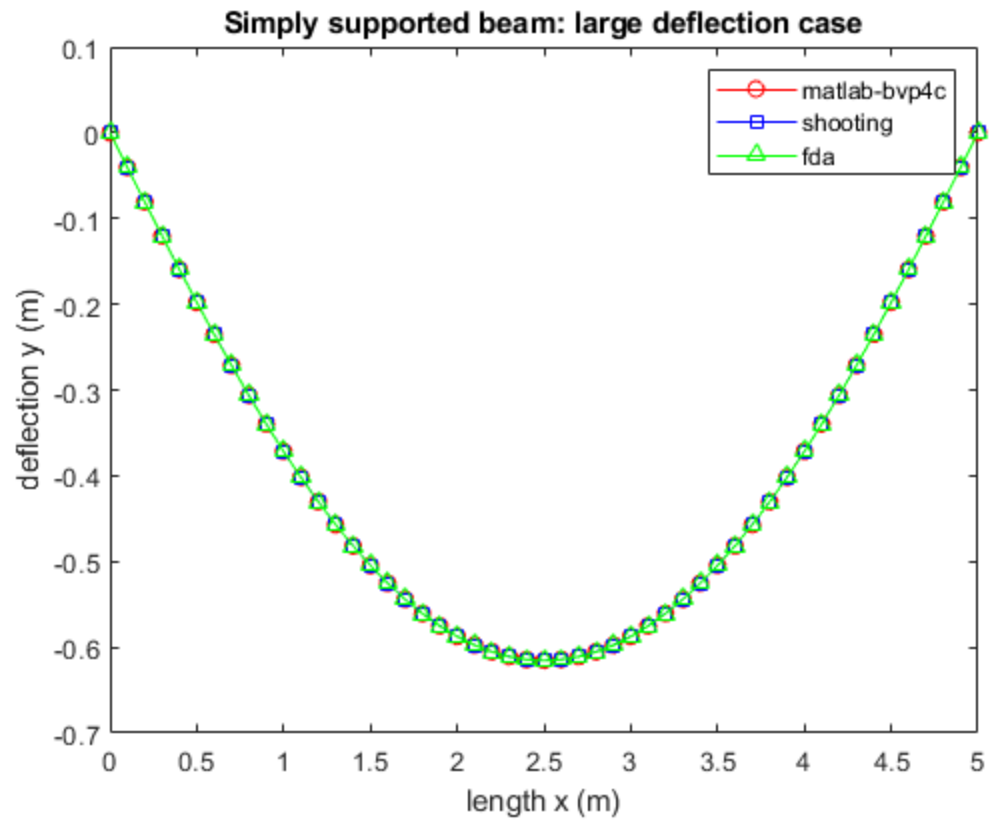
function [b] = beam_df(y,params)
n=length(y);
x=params{1};
y0=params{2};
yL=params{3};
b=zeros(n,1);
L = 5; q = 1500; E = 2e10; w = 0.05; h = 0.1;
I = h*w^3/12;
dx=x(2)-x(1);
qEI=q/(2*E*I);
for i=2:n-1
    b(i)=y(i-1)-2*y(i)+y(i+1)+qEI*dx^2*(1+((y(i+1)-y(i-1)))/(2*dx))^2)^1.5*(x(i)^2-L*x(i));
end
b(1)=y0-2*y(1)+y(2)+qEI*dx^2*(1+((y(2)-y0)/(2*dx))^2)^1.5*(x(1)^2-L*x(1));
b(n)=y(n-1)-2*y(n)+yL+qEI*dx^2*(1+((yL-y(n-1))/(2*dx))^2)^1.5*(x(n)^2-L*x(n));
return

```

Equation solved.

fsolve completed because the vector of function values is near zero as measured by the value of the function tolerance, and the problem appears regular as measured by the gradient.

number of iterations = 3
residual error = 5.38085e-05



Published with MATLAB® R2020a